20. Translating SOLID Principles into AI System Prompts

The 5 Architectural Pillars Preventing AI from Generating Spaghetti Disasters
Module 4 • Difficulty Level: Intermediate / Advanced • Estimated Reading Time: 13 min

---

> 🎙️ **NotebookLM & Audio Overview Podcast Briefing**
>
> **Podcast Discussion Hook:**
> *"SOLID principles were formulated decades ago for object-oriented systems; why are they the ultimate prompt constraints for AI code generation in 2026?"*
>
> **3 Core Audio Takeaways:**
> - 📌 Unconstrained AI models instinctively construct bloated 'God Objects' that handle validation, database queries, and notifications in one method. - 📌 Injecting SOLID principles directly into system prompts forces models to produce clean, decoupled classes and modules. - 📌 Dependency Inversion (DIP) is the single most crucial pattern for generating easily testable and mockable code with AI.

📖 1. The Story & Real-World Analogy

Examine a Swiss Army knife: it packs a blade, scissors, can opener, toothpick, and saw. It's handy for camping, but if you walked into an operating theater and saw a neurosurgeon wielding a Swiss Army knife, you'd run for your life.

A surgeon relies on dedicated, single-purpose scalpels.

Prompting an LLM with *"Implement user registration"* yields a bloated Swiss Army knife: one 300-line monster function that hashes passwords, mutates database tables, sends welcome emails, and generates invoice PDFs.

When you inject SOLID constraints into your prompts, the AI produces surgical scalpels: modular, single-responsibility units that never bleed into adjacent domains.

🧠 2. The AI Mental Model & Paradigm Shift

The SOLID to AI Prompt Translation Guide:

```
S - Single Responsibility: "Decompose this into discrete classes: separate database writes from mail notifications."
O - Open/Closed: "Design this via a strategy interface so novel payment gateways can be added without editing the core router."
L - Liskov Substitution: "Derived classes must satisfy base contracts without altering behavioral invariants or throwing unanticipated exceptions."
I - Interface Segregation: "Avoid bloated 20-method interfaces; split into composable, granular capabilities."
D - Dependency Inversion: "Inject dependencies via constructor interfaces; do not instantiate concrete database clients internally."
```

🤖 3. AI Prompt Lab (Production-Ready Templates)

You can directly paste and adapt this prompt template in Cursor, Claude Code, ChatGPT, or Copilot:

````
# ROLE: Clean Architecture Lead
Refactor this monolithic 'God Class' following SOLID principles:

## LEGACY SNIPPET:
```typescript
class OrderProcessor {
  async process(order: any) {
    if (order.amount <= 0) throw new Error("Invalid amount");
    await db.query(`INSERT INTO orders VALUES (${order.id}, ${order.amount})`);
    await mailer.send(order.email, "Order confirmed");
  }
}
````

TASK:

1. Apply Single Responsibility (SRP): decouple validation, persistence, and messaging into dedicated collaborators.
2. Apply Dependency Inversion (DIP): inject dependencies via constructor interfaces to enable mock unit testing.
````

````
---

## 💻 4. Code Walkthrough: Naive / Hallucination vs. Production-Grade

#### ❌ Naive / Unchecked AI Output (Risky):
```typescript
// SOLID'e Aykırı: Her işi tek sınıfta yapan God Object
export class OrderProcessorNaive {
  async processOrder(order: any) {
    // SRP ihlali: Validasyon, DB erişimi ve Mail tek yerde
    if (!order.id || order.amount <= 0) throw new Error("Geçersiz");
    // SQL Injection açığı da var!
    await db.query(`INSERT INTO orders VALUES ('${order.id}', ${order.amount})`);
    // DIP ihlali: Dış kütüphaneye doğrudan sıkı bağımlılık (tight coupling)
    await fetch("https://api.sendgrid.com/send", { body: JSON.stringify(order) });
  }
}
````
````

✅ **Senior-Refined Version (Robust & Clean):**

```
// SOLID Uyumlu: Modüler, Test Edilebilir, Tip Güvenli Mimari

// 1. Abstraksiyonlar (DIP)
export interface OrderRepository {
  save(order: ValidatedOrder): Promise<void>;
}

export interface NotificationService {
  notifyOrderPlaced(order: ValidatedOrder): Promise<void>;
}

export interface ValidatedOrder {
  readonly id: string;
  readonly amount: number;
  readonly customerEmail: string;
}

// 2. Orkestrasyon Sınıfı (SRP: Sadece iş akışını yönetir)
export class OrderProcessor {
  constructor(
    private readonly repository: OrderRepository,
    private readonly notifier: NotificationService
  ) {}

  async process(order: ValidatedOrder): Promise<void> {
    // Adım 1: Kaydet
    await this.repository.save(order);

    // Adım 2: Bildir
    await this.notifier.notifyOrderPlaced(order);
  }
}
```

💡 **Code Analysis:** `OrderProcessor` is completely decoupled from database drivers and third-party email SDKs. Unit testing this workflow requires only passing lightweight in-memory test doubles into the constructor.

⚠️ 5. Junior Pitfalls & Senior Perspective

- **Trap:** Over-engineering trivial 5-line scripts into 12 abstract factories.
  - **Remedy:** Apply SOLID pragmatically where domain rules shift or multiple implementations exist.
- **Trap:** Vaguely prompting ‘make it SOLID’ without specifying architectural intent.
  - **Remedy:** Pinpoint exact constraints: ‘Apply SRP to separate HTTP adapters from persistence’.
- **Trap:** Declaring interfaces yet instantiating concrete classes via `new` inside constructors.
  - **Remedy:** Always inject dependencies as constructor parameters to maintain Inversion of Control.

🎬 6. Udemy Video Lecture Blueprint & Slide Deck

📽️ Lecture Title: *Lecture 20: SOLID in Practice - Elevating AI Code to Enterprise Standards*

- 🖥️ **Slide 1:** Why unconstrained LLMs default to bloated God Objects.
- 🖥️ **Slide 2:** The 5 SOLID pillars explained through everyday engineering analogies.
- 🖥️ **Slide 3:** Decoupling systems via Dependency Inversion for testability.
- 🖥️ **Slide 4:** Live refactor: Slicing a 150-line procedural script into SOLID units.

Live Coding / Screencast Scenario:
Take an all-in-one procedural script. Prompt the AI: 'Refactor using SRP and Dependency Injection with TypeScript interfaces'. Show the resulting clean classes and write a mock unit test in 30 seconds.

🎯 7. Daily Action Task & Mini Quiz

🛠️ Daily Action Task:

Locate a monolithic function in your codebase (>100 lines). Extract logging or notification side-effects into a distinct injected collaborator.

🧠 Chapter Review Quiz:

Question 1: What is the formal definition of the Single Responsibility Principle (SRP)? - A) An engineering squad must have only one developer - B) A class or module should have one, and only one, reason to change - C) Every function must be exactly one line long - D) All variables must be single letters > **Correct Answer:** B > **Explanation:** SRP dictates that each class or function encapsulates one business capability, insulating it from unrelated requirement shifts.

Question 2: What is the primary architectural payoff of Dependency Inversion (DIP)? - A) Slowing down CPU instruction clocks - B) Decoupling high-level business rules from low-level I/O details, allowing frictionless unit test mocking - C) Exclusivity to Java enterprise applications - D) Automatic CSS flexbox formatting > **Correct Answer:** B > **Explanation:** DIP allows developers to substitute production infrastructure with in-memory test doubles during automated testing.